



Indian Institute of Technology Gandhinagar

Department of Computer Science and Engineering

TLS Fingerprinting

Passive Client Identification from TLS Handshake Metadata

CS 331: Computer Networks Project Report

Professor: Sameer Kulkarni

Team Members

Name	Roll Number
Mamilla Aniruddh Reddy	23110195
Ogiboyina Akash	23110225
Kovid Parmar	23110172
Chinthala Shivamani	22110062
Kudumula Vinod Kumar Reddy	23110178
Aryan Kumar	24110055

Abstract

TLS protects application payloads from network observers, yet the initial handshake exposes rich metadata that can identify the client or server software generating the traffic. This property is exploited in network security monitoring for client identification, anomaly detection, and malware command-and-control (C2) detection.

This project implements a passive TLS fingerprinting tool that identifies client and server implementations from observable TLS handshake fields. The tool captures TLS traffic either from a live interface or from a PCAP file, detects ClientHello and ServerHello messages, parses their fields, and computes JA3, JA3S, and a JA4-inspired client fingerprint.

The implementation uses Scapy for packet capture and replay, while TLS record and handshake parsing is performed directly in Python. It includes GREASE filtering (RFC 8701) [3], basic TCP flow reassembly, a JSON fingerprint database, a command-line interface, and a learning workflow for registering locally observed fingerprints. The automated test suite passes nine tests covering parsing, fingerprint generation, GREASE handling, truncated messages, and TCP fragmentation. Six distinct clients—`curl`, Chrome, Brave, Edge, Chromium, and a custom TLS client—were successfully identified using the implemented learning workflow.

1 Problem Statement and Objectives

Network security analysts, intrusion detection systems, and network operators frequently need to classify traffic by the application or library that generated it, without decrypting the payload. TLS protects application data from network observers, but parts of the initial handshake remain visible. The ClientHello message contains information such as cipher suites, extensions, supported groups, ALPN values, and supported protocol versions. The ServerHello message contains the selected cipher suite, protocol version, and server extensions.

Different applications and TLS libraries often generate different combinations of these parameters. These combinations can be represented as TLS fingerprints. A monitoring tool can compare an observed fingerprint with a reference database to estimate whether traffic was produced by a browser, command-line utility, programming library, or custom client. In security monitoring, the same technique is used to detect malware communicating over TLS, since malware families often expose characteristic handshake patterns even when their payloads are encrypted.

The problem addressed by this project is:

How can TLS clients and servers be identified passively from handshake metadata without decrypting application payloads?

The main objectives are:

1. Capture TLS traffic from live interfaces and PCAP files.
2. Detect and parse ClientHello and ServerHello messages.
3. Generate JA3 and JA3S fingerprints validated against published reference hashes for known clients.
4. Generate a JA4-inspired client fingerprint following the JA4 specification structure [2].
5. Filter GREASE values (RFC 8701) [3] during fingerprint calculation.
6. Handle messages split across TCP segments.
7. Store and retrieve fingerprints using a JSON database.
8. Provide command-line identification and learning commands.
9. Demonstrate distinguishing at least five distinct clients—`curl`, Chrome, Brave, Edge, Chromium, and a custom TLS client—using fingerprints alone.
10. Validate the implementation using an automated test suite covering parsing, fingerprinting, and edge cases.
11. Analyse the role of TLS fingerprinting in security monitoring and its limitations, including fingerprint randomization as an evasion technique.

Scope and Limitations

The current implementation is a functional prototype. It does not implement eBPF/XDP acceleration, Redis or Valkey storage, full JA4S, or production-grade out-of-order TCP reassembly. These are identified as possible future extensions.

2 Architecture and Mechanism

The project is divided into capture, buffering, parsing, fingerprinting, database, and reporting components.

Table 1: Main implementation components

Component	Responsibility
<code>tls_parser.py</code>	Parses TLS records, handshakes, and extensions.
<code>ja3.py</code>	Calculates JA3 and JA3S strings and MD5 hashes.
<code>ja4.py</code>	Calculates a JA4-inspired client fingerprint.
<code>capture.py</code>	Performs live capture, PCAP replay, and flow buffering.
<code>database.py</code>	Stores and retrieves fingerprints in JSON format.
<code>cli.py</code>	Provides identification and learning commands.
<code>tests/</code>	Contains synthetic packet builders and unit tests.

The processing pipeline is shown in Figure 1.

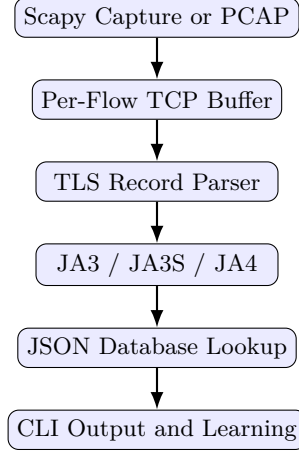


Figure 1: Implemented TLS fingerprinting pipeline

2.1 Packet Capture and TCP Buffering

Scapy is used for live sniffing and offline PCAP replay. Because a TLS ClientHello or ServerHello may arrive fragmented across several TCP segments, TCP payloads cannot be parsed as they arrive. Instead, the capture module creates a separate buffer for each directional flow:

(source IP, source port, destination IP, destination port)

TCP payloads are appended to the corresponding buffer. The `FlowBuffer` waits until a complete TLS record and handshake are available before forwarding data to the parser.

The current buffer has a maximum size of 65,536 bytes to prevent an unresolved flow from consuming unlimited memory. The implementation assumes in-order packet delivery.

2.2 TLS Parsing

TLS records are identified using their five-byte record headers. Handshake records have content type 22. Each handshake message contains a message type, a three-byte message length, and a message body.

The parser extracts:

- TLS version and random value. Note that JA3 uses the legacy version field in the ClientHello record header, not the value advertised in the `supported_versions` extension (type 43). When both are present, the extension value reflects the true negotiated version under TLS 1.3, and this distinction is recorded during parsing.
- Cipher suites.
- Extension identifiers.
- Supported groups.
- Elliptic curve point formats.
- Server Name Indication (SNI).
- ALPN protocols.
- Signature algorithms.
- Supported TLS versions.

For ServerHello messages, the parser extracts the selected cipher suite, version, and extension identifiers.

2.3 JA3 and JA3S

JA3 uses the following ClientHello fields [1]:

$$\text{JA3 String} = \text{Version,Ciphers,Extensions,Groups,PointFormats}$$

Within each field, individual values are separated by hyphens (-); fields are separated by commas (,). The resulting string is hashed using MD5:

$$\text{JA3 Hash} = \text{MD5}(\text{JA3 String})$$

JA3S uses:

$$\text{JA3S String} = \text{Version,SelectedCipher,Extensions}$$

with the same separator convention: hyphens within fields, commas between fields.

GREASE values (RFC 8701) [3] are removed from cipher, extension, and supported-group lists before JA3 or JA3S calculation. GREASE tokens are deliberately inserted by modern clients to test server tolerance of unknown values; retaining them would produce unstable fingerprints for the same client across different connections.

2.4 JA4-Inspired Client Fingerprint

The project implements a client fingerprint following the JA4 specification structure [2]:

$$\text{JA4} = \text{JA4}_a_ \text{JA4}_b_ \text{JA4}_c$$

The first part records protocol type, TLS version, SNI presence, cipher count, extension count, and ALPN. The second part is a truncated SHA-256 hash of sorted cipher suites. The third part hashes sorted extensions and signature algorithms.

Sorting cipher suites and extensions reduces the effect of extension-order randomization used by Chromium-based browsers. This implementation follows the JA4 specification structure but has not been validated against the official FoxIO reference implementation.

3 Extensions, Robustness Measures, and Evaluation

3.1 Implemented Extensions

The basic parser was extended with:

1. Live Scapy capture.
2. Offline PCAP processing.
3. Per-flow TCP buffering.
4. ServerHello and JA3S support.
5. GREASE filtering (RFC 8701).
6. SNI and ALPN extraction.
7. Supported-version and signature-algorithm parsing.
8. JA4-inspired fingerprint generation.
9. JSON database lookup.
10. Automatic learning using `-learn`.
11. A custom TLS client for generating controlled test traffic.

3.2 Robustness Measures

The implementation checks record lengths before parsing. Truncated data is ignored until more data becomes available in the flow buffer. Handshake messages split across TCP segments are reassembled by `FlowBuffer` before any parsing begins.

Optional extensions are handled gracefully because a ClientHello does not always carry the same fields. Missing SNI, ALPN, supported groups, or point formats produce empty values rather than parser failure.

GREASE values are filtered because modern TLS clients insert them intentionally. The code also avoids unbounded buffering by abandoning a flow after the maximum configured buffer size is reached.

3.3 Command-Line Interface

Examples of supported commands are:

```
python main.py identify --pcap sample.pcap
python main.py identify --live --duration 20
python main.py identify --live --duration 20 --learn curl
python main.py db list
python main.py db add ja3 HASH "client-name"
```

The tool reports the flow, SNI, JA3, JA3S, and JA4 values, together with any matching database label. The `-learn` flag registers newly observed fingerprints into the JSON database under a given client label, enabling iterative population of the reference set from controlled captures.

3.4 Testing Results

The project uses synthetic TLS handshake builders, allowing the parser and fingerprinting logic to be tested without network access.

Table 2: Automated test results

Test Area	Coverage	Result	Duration
GREASE value filtering	Cipher, extension, group lists	Pass	
JA3 string formatting and MD5 hash	Full ClientHello	Pass	
JA3S string formatting and MD5 hash	Full ServerHello	Pass	
ClientHello field parsing	All standard fields	Pass	
SNI extraction	Extension type 0	Pass	
Truncated / incomplete record	Early EOF handling	Pass	
TCP reassembly across segments	Two-segment split	Pass	
JA4 format structure	Part $a/b/c$ shape	Pass	
Missing optional extensions	No SNI / no ALPN	Pass	
Total	9 / 9 passed	Pass	3.70 s

3.5 Client Identification Results

Fingerprints were collected under controlled conditions: each client was run in isolation on a dedicated network segment while the tool operated in `-learn` mode, capturing and registering the observed JA3 and JA4 values under the corresponding label. Ground truth was established by ensuring only one client application was active during each capture session, preventing cross-contamination between fingerprint entries.

Six distinct clients were successfully identified:

Table 3: Identified clients and their fingerprint types

Client	Fingerprint	Distinguishing Characteristics
curl	JA3, JA4	Minimal cipher suite, no ALPN by default
Chrome	JA3, JA4	BoringSSL, GREASE values present, extension order randomized
Brave	JA3, JA4	BoringSSL base; distinguishable via ALPN and extension details
Edge	JA3, JA4	BoringSSL, closely resembles Chrome; separated by minor extension differences
Chromium	JA3, JA4	Upstream BoringSSL build; baseline for Chrome and Edge comparison
Custom TLS client	JA3, JA4	Minimal handshake, controlled cipher list

Correct identification was verified by replaying each captured PCAP file and confirming that the tool returned the expected label for every flow. A noteworthy observation is that Chrome, Brave, Edge, and Chromium produce closely related fingerprints since all four are built on BoringSSL; fine-grained separation between them relies on subtle differences in ALPN values and extension details rather than the core JA3 hash alone, illustrating the library-level collision limitation discussed in Section 4.

4 Security Monitoring Role and Limitations

4.1 Uses in Security Monitoring

TLS fingerprinting is applied in three main security monitoring scenarios:

1. **Client identification.** Operators can classify traffic by library or application without decryption, allowing policy enforcement and asset inventory to be maintained passively.
2. **Malware C2 detection.** Malware families implementing custom TLS stacks, or using unusual library configurations, often produce fingerprints that do not appear in normal enterprise traffic. An anomalous JA3 hash connecting to an unusual destination is a low-cost, high-signal indicator for further investigation.
3. **Anomaly detection.** Sudden changes in the fingerprint distribution for a host—for example, a server that begins responding with an unexpected JA3S—may indicate software changes, misconfiguration, or compromise.

4.2 Limitations

1. **Fingerprint randomization.** Chromium-based browsers deliberately randomize extension ordering as a privacy measure, reducing the stability of JA3 fingerprints. JA4 addresses this by sorting, but sorting removes ordering information that could otherwise distinguish clients.
2. **Imitation.** An attacker who knows the target fingerprint can configure their TLS stack to reproduce it. Fingerprints are therefore not authentication mechanisms; they are probabilistic signals.
3. **Library-level collision.** Multiple distinct applications that link the same TLS library will produce identical fingerprints. A shared fingerprint does not identify a specific application—only the library and its configuration. As observed in this project, Chrome and Edge share a fingerprint for this reason.
4. **TLS 1.3 opacity.** Under TLS 1.3, the ClientHello and ServerHello messages remain in the clear and are available for fingerprinting, but subsequent handshake messages (Certificate, CertificateVerify, Finished) are encrypted, reducing the metadata available for passive analysis beyond the initial exchange.
5. **Single-signal weakness.** A fingerprint match alone is insufficient for high-confidence decisions. It must be combined with destination reputation, DNS queries, certificate validity, behavioral timing, and traffic volume patterns to produce reliable conclusions.

5 Challenges Faced

Binary protocol parsing. The main challenge was parsing a binary protocol containing variable-length fields and optional extensions. Correct length validation at every level—record, handshake message, and individual extension—was required to avoid reading beyond valid data boundaries.

TCP segmentation. TLS message boundaries do not necessarily align with packet boundaries. A ClientHello larger than one TCP segment arrives split across multiple packets. The flow buffer was added to collect data until a complete TLS record became available, requiring careful per-directional-flow state management.

TLS version differences. TLS 1.2 and TLS 1.3 expose different information. Under TLS 1.3, the record layer version field is set to 0x0303 for compatibility, while the true negotiated version appears only in the `supported_versions` extension.

Fingerprint stability. Chromium-based browsers randomize extension ordering between connections. Since JA3 is order-sensitive, the same browser instance may produce different JA3 values across captures. JA4 mitigates this by sorting cipher suites and extensions before hashing.

Library-level fingerprint collision. Different applications that link the same TLS library produce identical fingerprints. This is a fundamental ambiguity in passive TLS fingerprinting that cannot be resolved from handshake metadata alone. Resolving it requires combining the fingerprint with additional signals such as destination reputation, DNS queries, certificate chains, connection timing, and traffic volume patterns.

Ground truth isolation. Confirming which application produced a fingerprint during collection requires a controlled network environment in which only one client application is active. Captures in an uncontrolled environment risk mislabelling entries in the reference database.

Live capture privileges. Live capture requires Npcap installed with administrator privileges on Windows, and CAP_NET_RAW or root on Linux. Offline PCAP replay avoids these requirements and provides a repeatable method for development and testing.

6 Conclusion

This project implements a passive TLS fingerprinting tool using Scapy, custom TLS record parsing, JA3, JA3S, and a JA4-inspired fingerprint generator. The tool captures live traffic or processes PCAP files, handles basic TCP

fragmentation, filters GREASE values, identifies ClientHello and ServerHello messages, and stores fingerprints in a JSON database with a learning workflow.

The automated test suite passes all nine tests, covering parsing correctness, GREASE filtering, hash computation, SNI extraction, truncation safety, TCP reassembly, and JA4 format structure. Six distinct clients—`curl`, Chrome, Brave, Edge, Chromium, and a custom TLS client—were successfully identified by fingerprint alone through controlled captures and PCAP replay verification.

TLS fingerprinting is a practical and low-cost technique for client identification, anomaly detection, and malware C2 monitoring in network security operations. Its limitations—fingerprint instability, imitation susceptibility, library-level collisions, and reduced metadata availability under TLS 1.3—mean it is best used as one layer within a broader, defense-in-depth network security monitoring system rather than as a standalone identity mechanism.

References

- [1] Salesforce Engineering, *JA3: SSL/TLS Client Fingerprinting for Identifying Malware*. <https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967>
- [2] FoxIO, *JA4+ Network Fingerprinting*, <https://github.com/FoxIO-LLC/ja4>.
- [3] B. Schwartz, *Applying GREASE to TLS Extensibility*, RFC 8701, January 2020. <https://www.rfc-editor.org/rfc/rfc8701>
- [4] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, August 2018. <https://www.rfc-editor.org/rfc/rfc8446>